Practice Guide 1: A Spam Filter from Scratch (Naive Bayes, TF-IDF, Logistic Regression)

Aprendizagem Aplicada à Segurança

Mário Antunes

September 25, 2026

Objective

Build, **by yourself**, the first rungs of the spam ladder presented in the lecture: a tokenizer, a **Naive Bayes** classifier that works in **log space**, the **evaluation metrics**, **TF-IDF**, and a **logistic regression** trained by gradient descent with JAX. At the end you will have a filter that reaches an F1 of about 0.95 on unseen messages, and you will know *why* each ingredient is there.

- **Notebook to complete:** `guide_02_01.ipynb` (this folder). Cells marked `TODO` are yours; `check` cells tell you whether you are right.
- **Data:** `datasets/spam.csv` (SMS Spam Collection: 5,574 messages, about 13% spam).
- **Time:** about 2 hours (suggested: A to C 15 min, D 25, E 10, F 15, G 5, H 30, I 10).
- **Environment:** `make venv` at the root of the repository (see `pyproject.toml`); JAX is installed with it.

Background in one page

Bayes' theorem for text. For a message with words $W_1, \ldots, W_n$ and class $y \in \{\text{ham}, \text{spam}\}$:

$$P(y \mid W_1..W_n) = \frac{P(y) \prod_i P(W_i \mid y)}{\sum_{y'} P(y') \prod_i P(W_i \mid y')}$$

The product assumes that words are independent given the class (the *naive* assumption). With word counts x_w and Laplace smoothing k:

$$P(w \mid y) = \frac{N_{w,y} + k}{\sum_{w'} N_{w',y} + k|V|}$$

Why logarithms. A long message multiplies hundreds of small probabilities and the result underflows to 0. Work with $\ell_y = \log P(y) + \sum_w x_w \log P(w \mid y)$ and normalise with **log-sum-exp**: $\log \sum_y e^{\ell_y} = m + \log \sum_y e^{\ell_y - m}$, $m = \max_y \ell_y$.

Metrics. Spam is the positive class. precision $= \frac{TP}{TP+FP}$, recall $= \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$.

TF-IDF. tf $= 1 + \ln c_{t,d}$, idf$(t) = \ln \frac{1+N}{1+\text{df}(t)} + 1$, then L2-normalise each message.

Logistic regression. $z = \mathbf{w} \cdot \mathbf{x} + b$, $P(\text{spam}) = \sigma(z)$. Cross-entropy with L2: $J = -\frac{1}{n} \sum_i [y_i \log \sigma(z_i) + (1 - y_i) \log \sigma(-z_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$, gradient $\frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda \mathbf{w}$. Never compute `log(sigmoid(z))` directly: for $z = -800$ it is $-\infty$; use `jax.nn.log_sigmoid`. Gradient descent: $\theta \leftarrow \theta - \eta \nabla J$.

Tasks

Part	What you do	Check
A	Load the data, remove duplicates, stratified split (given)	Question A1
B	Implement `tokenize`	`tokenizer OK`
C	Count matrix over the training vocabulary (given)	
D	Implement `NaiveBayes` (fit, joint log-probability, log-sum-exp)	matches scikit-learn
E	Confusion counts, precision, recall, F1; choose an operating point	`metrics OK`
F	Implement TF-IDF	matches scikit-learn
G	Compare counts and TF-IDF, $k = 0.1$ and $k = 1$ (the split repetition is optional)	comparison table
H	**Logistic regression in JAX:** stable loss, one gradient-descent step, loss curves for three learning rates	`loss and gradient OK`, Question H1
I	Read the learned words of Naive Bayes and of the logistic regression	Questions I1, I2

Tips. Do not use loops over the messages when a matrix product will do. Run the `check` cell after each part. If a check fails, print the shapes and a few values before changing code.

Questions

1. **(A1)** Why do we split the data *before* building the vocabulary, and why `stratify=y`? What could go wrong with the duplicated messages?
2. **(B)** Which token classes did we keep on purpose (`£`, `!`, `num`)? Give one example of a spam message that would be harder to detect without them.
3. **(D1)** Report the posterior of the 1,000-word message computed with and without logarithms. Explain the difference.
4. **(D2)** Change k to 0.01 and to 10. How do the probabilities and the F1 change? Why does a large k make the model less sure?
5. **(E1)** What accuracy, precision and recall does a filter that always answers *ham* obtain? Which metric exposes it?
6. **(E2)** Report the threshold that gives precision ≥ 0.99 and the recall you pay for it. Who prefers this operating point, and why?
7. **(F1)** Why do `you`, `to`, `the` receive the lowest idf? What does the L2 normalisation change for a very long message?
8. **(G)** Fill in the table below. Is TF-IDF better than counts for Naive Bayes on this corpus? (Optional: repeat with 3 values of `random_state` and report the spread before answering.)
9. **(H1)** Plot the loss history of the logistic regression for $\eta = 1, 50$ and 500. Describe and explain each behaviour.
10. **(H2)** Report precision, recall and F1 of the logistic regression next to the best Naive Bayes. Which has the higher precision, which the higher recall, and what does that mean for a mail gateway?
11. **(I1)** `lt` and `gt` appear among the "hammy" words. Where do they come from? What does this tell you about the model, and how could an attacker use the list of most hammy words?
12. **(I2)** The weight vector of the logistic regression is the gradient of its score with respect to the input. What would an attacker need to know, and do, to use it against the filter?

Representation	k	Precision	Recall	F1
counts	0.1			
counts	1			
TF-IDF	0.1			
TF-IDF	1			
logistic regression (TF-IDF, GD)				

Optional challenges

- Add **bigrams** (`call now`, `free entry`) to the vocabulary. What happens to the vocabulary size and to the score?
- Add a **length** feature (number of tokens) and a **digit-count** feature. Do they help?
- Implement a χ^2 feature selection and plot the F1 against the vocabulary size.

Reference solution

The solved notebook is `solution/guide_02_01.ipynb`.